

- 1) Com base no código a seguir, desenhe as variáveis e seus conteúdos finais.

```
double valor;  
char letra;  
char *p1;  
double *p2;
```

```
valor = 10.25;  
letra = 'C';
```

```
p1 = &letra;  
p2 = &valor;
```

```
*p1 = 'H';
```

- 2) Suponha que um programa declara as variáveis e faz atribuições conforme as instruções a seguir. Desenhe as variáveis com os seus respectivos conteúdos finais.

```
int comprimento, *pin;  
float altura, *pfl;  
struct regDados {int quantidade; float valor;} vDado, *pDado;
```

```
comprimento = 25;
```

```
pfl = &altura;  
*pfl = 32.87;
```

```
pDado = &vDado;  
pin = &vDado.quantidade;  
pfl = &vDado.valor;  
vDado.quantidade = 15;  
pDado->valor = (*pDado).quantidade * 4.85;
```

- 3) Considerando as definições em C apresentadas a seguir, faça testes em programas e responda às questões formuladas.

```
float valores[10], *p;
```

```
p = &valores[0];  
printf("%p %p\n", valores, p);
```

```
p = valores;  
printf("%p %p\n", valores, p);
```

```
p = &valores[1];  
printf("%x %p\n", p - valores, p);
```

```
p = &valores[6];
```

ATIVIDADE PONTEIROS

Aula de Revisão

RESOLUÇÃO

- 1) Com base no código a seguir, desenhe as variáveis e seus conteúdos finais.

```
double valor;
```

```
char letra;
```

```
char *p1;
```

```
double *p2;
```

```
valor = 10.25;
```

```
letra = 'C';
```

```
p1 = &letra;
```

```
p2 = &valor;
```

```
*p1 = 'H';
```

RESOLUÇÃO

- 1) Com base no código a seguir, desenhe as variáveis e seus conteúdos finais.

```
double valor;
```

```
char letra;
```

```
char *p1;
```

```
double *p2;
```

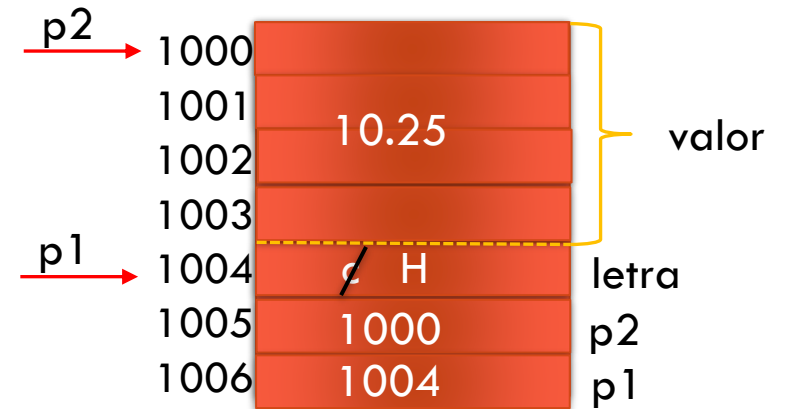
```
valor = 10.25;
```

```
letra = 'C';
```

```
p1 = &letra;
```

```
p2 = &valor;
```

```
*p1 = 'H';
```



RESOLUÇÃO

- 2) Suponha que um programa declara as variáveis e faz atribuições conforme as instruções a seguir. Desenhe as variáveis com os seus respectivos conteúdos finais.

```
int comprimento, *pin;
```

```
float altura, *pfl;
```

```
struct regDados {int quantidade; float valor;} vDado, *pDado;
```

```
comprimento = 25;
```

```
pfl = &altura;
```

```
*pfl = 32.87;
```

```
pDado = &vDado;
```

```
pin = &vDado.quantidade;
```

```
pfl = &vDado.valor;
```

```
vDado.quantidade = 15;
```

```
pDado->valor = (*pDado).quantidade * 4.85;
```

RESOLUÇÃO

- 2) Suponha que um programa declara as variáveis e faz atribuições conforme as instruções a seguir. Desenhe as variáveis com os seus respectivos conteúdos finais.

```
int comprimento, *pin;  
float altura, *pfl;  
struct regDados {int quantidade; float valor;} vDado, *pDado;
```

```
comprimento = 25;
```

```
pfl = &altura;
```

```
*pfl = 32.87;
```

```
pDado = &vDado;
```

```
pin = &vDado.quantidade;
```

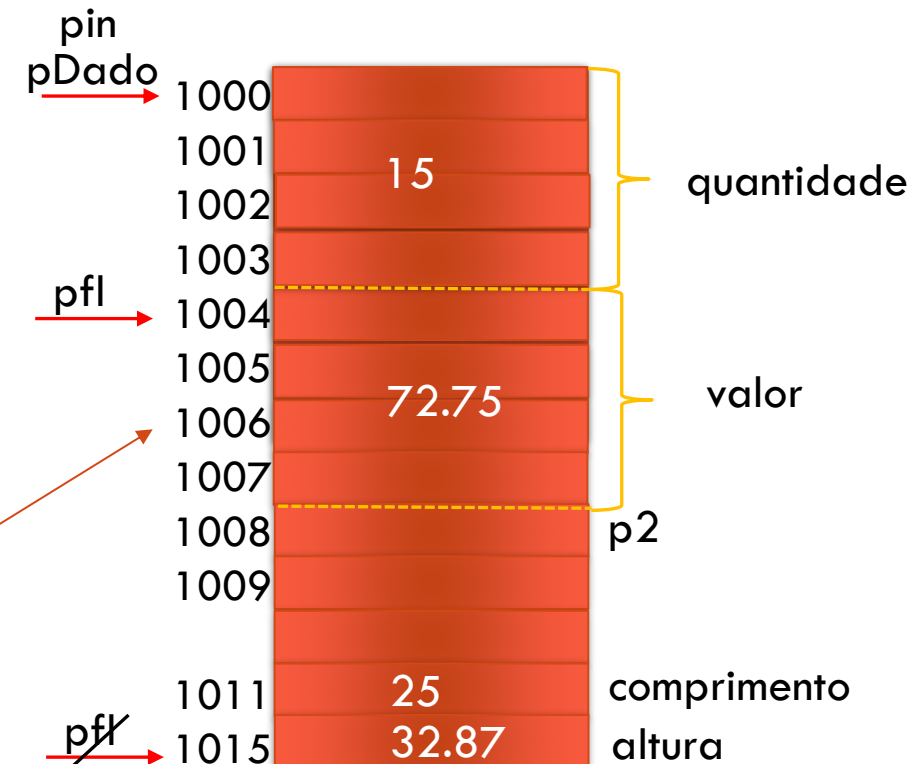
```
pfl = &vDado.valor;
```

```
vDado.quantidade = 15;
```

```
pDado->valor = (*pDado).quantidade * 4.85;
```

Como `vDado.quantidade = 15` logo
`(*pDado).quantidade` será 15
 $15 * 4.85 = 72.75$

guardar 72.75 em `pDado->valor`



RESOLUÇÃO

- a) O que significa mencionar, no código de um programa, apenas o nome de um vetor já declarado?
- b) O que acontece se, ao final das instruções anteriores, fizermos $(*p) = 15$?
- c) Qual o significado do resultado impresso no primeiro valor do último `printf()` do código?

3) Considerando as definições em C apresentadas a seguir, faça testes em programas e responda às questões formuladas.

```
float valores[10], *p;
```

```
p = &valores[0];
```

```
printf("%p %p\n", valores, p);
```

```
p = valores;
```

```
printf("%p %p\n", valores, p);
```

```
p = &valores[1];
```

```
printf("%x %p\n", p - valores, p);
```

```
p = &valores[6];
```

```
printf("%x %p\n", p - valores, p);
```

RESOLUÇÃO

- a) O que significa mencionar, no código de um programa, apenas o nome de um vetor já declarado? **Corresponde ao endereço do primeiro elemento do vetor**
- b) O que acontece se, ao final das instruções anteriores, fizermos $(*p) = 15$?
O sétimo espaço do vetor, valores[6] passará a conter o valor 15
- c) Qual o significado do resultado impresso no primeiro valor do último `printf()` do código? **Mostrará o valor 6 e o end do sétimo valor**

3) Considerando as definições em C apresentadas a seguir, faça testes em programas e responda às questões formuladas.

```
float valores[10], *p;
```

```
p = &valores[0];
```

← Apontando para o primeiro espaço do vetor

```
printf("%p %p\n", valores, p);
```

← Mostra o mesmo endereço 2x

```
p = valores;
```

← Apontando para o end mais baixo do vetor

```
printf("%p %p\n", valores, p);
```

← Mostra o mesmo endereço 2x

```
p = &valores[1];
```

← Apontando para o segundo espaço do vetor

```
printf("%x %p\n", p - valores, p);
```

← &valor[0]-&valor[1], diferença entre índice, logo 1
 ← End de valores[1]

```
p = &valores[6];
```

← Apontando para o sétimo espaço do vetor

```
printf("%x %p\n", p - valores, p);
```

← 6 corresponde o deslocamento (índ) em relação ao início do vetor e End de valores[6]

RESOLUÇÃO

4) [Questão 38 do Poscomp 2007] Dado o seguinte programa escrito em C:

```
#include <stdio.h>

int main(void)
{   int n[] = {7, 8, 9};
    int *p;
    p = &n[0];
    p++;
    printf("Valor: %d ", *p);
    (*p)++;
    printf("Valor: %d\n", *p);
}
```

Qual é a resposta que será impressa na tela:

- a) Valor: 7 Valor : 8
- b) Valor: 7 Valor: 7
- c) Valor: 8 Valor: 9
- d) Valor: 7 Valor: 9
- e) Valor: 9 Valor: 9.

RESOLUÇÃO

4) [Questão 38 do Poscomp 2007] Dado o seguinte programa escrito em C:

```
#include <stdio.h>
```

```
int main(void)
```

```
{  int n[] = {7, 8, 9};
```

```
    int *p;
```

```
    p = &n[0];
```

← Apontando para o primeiro espaço do vetor

```
    p++;
```

← Apontando para o segundo espaço do vetor

```
    printf("Valor: %d ", *p);
```

```
    (*p)++;
```

← Incrementando o valor 8

```
    printf("Valor: %d\n", *p);
```

← Mostrando o 9

```
}
```

Qual é a resposta que será impressa na tela:

a) Valor: 7 Valor : 8

b) Valor: 7 Valor: 7

c) Valor: 8 Valor: 9

d) Valor: 7 Valor: 9

e) Valor: 9 Valor: 9.

RESOLUÇÃO

- 5) Considere as variáveis declaradas no trecho de código a seguir. Faça um desenho ilustrando essas variáveis e seus conteúdos na situação apresentada e depois responda às questões formuladas.

```
char texto[25], palavra[15], descr[11], *p1, *p2;
strcpy(palavra, "xxxxxxx");
strcpy(texto, "Linguagem C");
scanf("%s", &palavra[3]);
p1 = &texto[6];
p2 = &descr[0];
scanf("%s", p2);          /* O usuário digita "goiaba" aqui */
strcpy(&texto[6], " de trapo");
```

- a) Considerando que no primeiro `scanf()` o usuário digitou “Banana”, qual o conteúdo da variável `palavra` após essa entrada de dados? Qual seria o resultado da instrução `strlen(palavra)` nesse caso? E qual o resultado para `strlen(&palavra[3])`?
- b) Qual o conteúdo da variável `descr`?
- c) Por que a chamada a `scanf()` não tem ‘&’ quando o conteúdo digitado destina-se a uma *string*?

RESOLUÇÃO

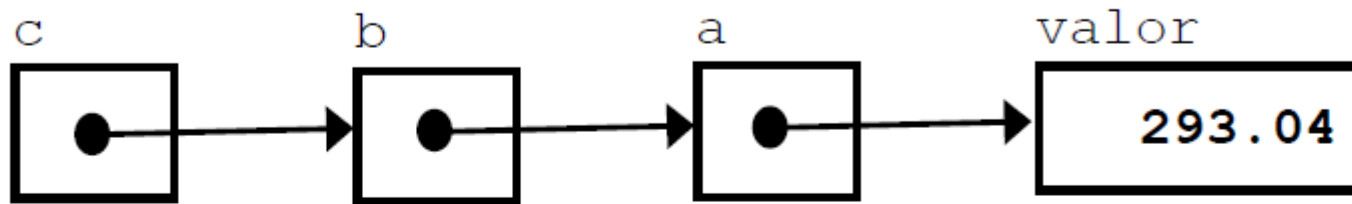
- 5) Considere as variáveis declaradas no trecho de código a seguir. Faça um desenho ilustrando essas variáveis e seus conteúdos na situação apresentada e depois responda às questões formuladas.

```
char texto[25], palavra[15], descr[11], *p1, *p2;
strcpy(palavra, "xxxxxxx"); ← palavra é xxxxxxxx
strcpy(texto, "Linguagem C"); ← texto é Linguagem C
scanf("%s", &palavra[3]); ← xxxBanana e strlen(palavra)=9, strlen(&palavra[3])=6
p1 = &texto[6];
p2 = &descr[0]; ← O que for digitado no segundo scanf(), no caso goiaba
scanf("%s", p2); /* O usuário digita "goiaba" aqui */
strcpy(&texto[6], " de trapo");
```

- a) Considerando que no primeiro `scanf()` o usuário digitou “Banana”, qual o conteúdo da variável `palavra` após essa entrada de dados? Qual seria o resultado da instrução `strlen(palavra)` nesse caso? E qual o resultado para `strlen(&palavra[3])`?
- b) Qual o conteúdo da variável `descr`? **goiaba**
- c) Por que a chamada a `scanf()` não tem ‘&’ quando o conteúdo digitado destina-se a uma *string*? **Porque estamos recebendo direto no ponteiro ou local do endereço**

RESOLUÇÃO

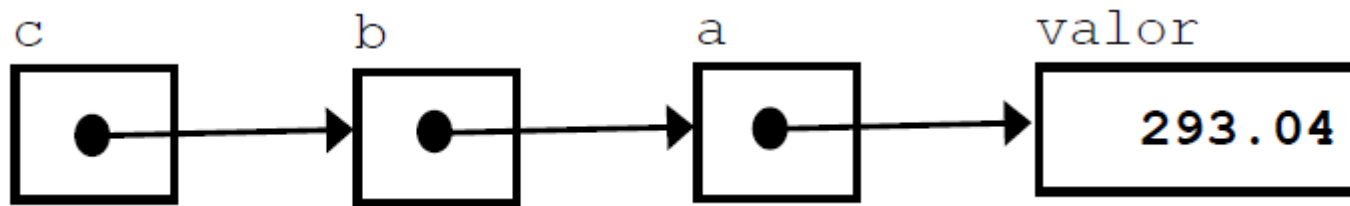
- 6) Escreva as instruções necessárias em linguagem C para produzir a situação apresentada na figura.



- 7) Suponha que temos uma subrotina que recebeu como seu primeiro parâmetro um vetor, cujos elementos são do tipo `struct regDados`, e um valor a ser pesquisado nesse vetor como sendo o seu segundo parâmetro. Após feita a pesquisa, o valor foi encontrado, e temos o ponteiro `x` indicando o endereço de memória desse elemento que contém o dado desejado. Qual a instrução que calcularia quantos bytes existem entre o início do *array* na memória e o início do elemento onde o valor foi encontrado? Assuma que o nome do primeiro parâmetro formal é `v`.

RESOLUÇÃO

- 6) Escreva as instruções necessárias em linguagem C para produzir a situação apresentada na figura.



```
float a = 293.04;
```

```
float *b = &a;
```

```
float **c = &b;
```

```
printf("Valor: %.2f\n", **c);
```

RESOLUÇÃO

- 7) Suponha que temos uma subrotina que recebeu como seu primeiro parâmetro um vetor, cujos elementos são do tipo `struct regDados`, e um valor a ser pesquisado nesse vetor como sendo o seu segundo parâmetro. Após feita a pesquisa, o valor foi encontrado, e temos o ponteiro `x` indicando o endereço de memória desse elemento que contém o dado desejado. Qual a instrução que calcularia quantos bytes existem entre o início do *array* na memória e o início do elemento onde o valor foi encontrado? Assuma que o nome do primeiro parâmetro formal é `v`.

O vetor é `struct regDados v[]`

Se `pbusca` aponta para um elemento onde foi encontrado um valor, logo

$(pbusca - v) * \text{sizeof}(\text{struct regDados})$, calculará quantos bytes existem entre o início do vetor e esse elemento.

EXERCÍCIOS

Utilizar ponteiro para manipular a vetor.



Ex1 - Ler e armazenar 2 notas, verificar se estão no intervalo de 0 a 10. Calcular a média para até 100 alunos e verificar:

- se $m \geq 7$ – imprimir **APROVADO**
- se $4 \leq m < 7$ – imprimir **EM RECUPERAÇÃO**
- se $m < 4$ – imprimir **REPROVADO**

Ex2 - Receba uma frase e uma letra do usuário, conte quantas ocorrências da letra existem na frase, bem como as suas posições.

EXERCÍCIOS

```
struct cliente{  
    char nome[30];  
    int conta;  
    float saldo;  
};
```

Ex 3 - Fazer um programa para **Sistema de Conta Bancária** – este programa se destina a controlar as contas de clientes. Consultar o usuário para continuar.
Utilizar ponteiros para a estrutura.

[1] Cadastro - receber os valores digitados pelo usuário. Apenas um registro é cadastrado por vez.

[2] Depósito - o acesso deve ser feito através do nº de conta corrente. Buscar o registro, mostrar o nome do cliente e o saldo para simples conferência, pedir o valor do depósito, fazer as alterações e apresentar na tela o saldo atualizado.

[3] Retirada - o acesso deve ser feito através do nº de conta corrente. Buscar o registro, mostrar o nome do cliente e o saldo para simples conferência, pedir o valor da retirada, fazer as alterações se possível (a retirada só será permitida, se houver saldo suficiente) e apresentar na tela o saldo atualizado.

DÚVIDAS

- Questionamentos?

